CHƯƠNG IV. TRUY VẤN KÝ TỰ ĐẠI DIỆN VÀ SỬA LỖI CHÍNH TẢ

Truy xuất thông tin

ThS. Phạm Đức Cường
cuongpd@ptit.edu.vn

Posts and Telecommunications
Institute of Technology

Học viện Công nghệ Bưu chính Viễn thông

Nội dung

1. Truy vấn ký tự đại diện
2. Sửa lỗi chính tả

Truy vấn ký tự đại diện: * (wild-cards)

- **mon***: tìm tất cả tài liệu chứa **bất kỳ từ nào bắt đầu bằng "mon"**.
  - Làm dễ dàng với từ điển dạng **cây nhị phân** (**B-tree**): lấy tất cả các từ trong khoảng: **mon ≤ w < moo**
- ***mon**: tìm các từ **kết thúc bằng "mon"**: khó hơn.
  - Duy trì thêm một **B-tree** cho các thuật ngữ **được đảo ngược ký tự** (viết ngược). Khi đó có thể lấy các từ trong khoảng: **nom ≤ w < non**

Từ đây, làm thế nào để **liệt kê (enumerate) tất cả các thuật ngữ** thỏa truy vấn wild-card **pro*cent**?

Xử lý truy vấn

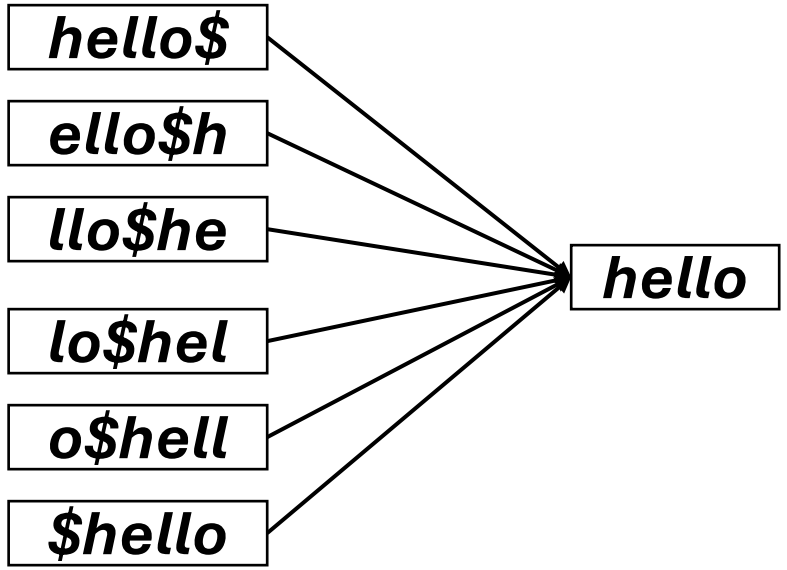
- **Tại thời điểm này**, chúng ta đã **liệt kê được tất cả các thuật ngữ trong từ điển** khớp với truy vấn **wild-card**.
- Tuy nhiên, chúng ta vẫn cần **tra cứu danh sách postings cho mỗi thuật ngữ đã liệt kê**.
- **Ví dụ**, xét truy vấn:
  - ***se*ate*** *AND* ***fil*er***
  - Truy vấn này có thể dẫn đến việc thực hiện **rất nhiều truy vấn Boolean AND**.

B-trees xử lý dấu * ở cuối từ truy vấn

- **B-trees** có thể xử lý dễ dàng dấu * **ở cuối một từ trong truy vấn**.
- Nhưng làm thế nào để xử lý dấu * **ở giữa từ truy vấn**?
- Ví dụ:
  - ***co*tion***
- Một cách làm là tìm kiếm trong **B-tree**:
  - ***co**** AND ***tion***
- Sau đó **lấy giao của hai tập thuật ngữ**.
  - Tuy nhiên cách này **tốn kém chi phí tính toán**.
- **Giải pháp**
  - Biến đổi truy vấn wild-card sao cho dấu * **luôn nằm ở cuối từ**.
  - Cách này dẫn tới cấu trúc dữ liệu gọi là **Permuterm** Index

Permuterm index

- Thêm ký hiệu đặc biệt ***$ (ký hiệu đặc biệt đánh dấu ranh giới từ)*** vào cuối mỗi từ trong từ điển.
- Sau đó **xoay vòng từ đó** và **lập chỉ mục các phiên bản xoay** trong một **B-tree**.
- Ví dụ với từ ***hello:***
  - Thêm ký hiệu: ***$***
  - Sau đó tạo các bản xoay vòng và lập chỉ mục
  - ***hello$, ello$h, llohe, lohel, o$hell, $hello***



Trong thực tế, **kích thước từ điển tăng khoảng 4 lần**

Xử lý truy vấn Permuterm

- **Thêm ký hiệu $.**
- **Xoay vòng** để đưa ký tự * **ra cuối chuỗi.**
- **Tra cứu trong permuterm index.**

Truy vấn	Cách tra cứu trong permuterm index	Ví dụ
X	tra cứu X$	hello → hello$
X*	tra cứu $X*	hel* → $hel*
*X	tra cứu X$*	*llo → llo$*
X	tra cứu X*	*ell* → ell*
X*Y	tra cứu Y$X*	h*lo → lo$h*
X*Y*Z	tìm X*Z rồi **lọc lại kết quả**	

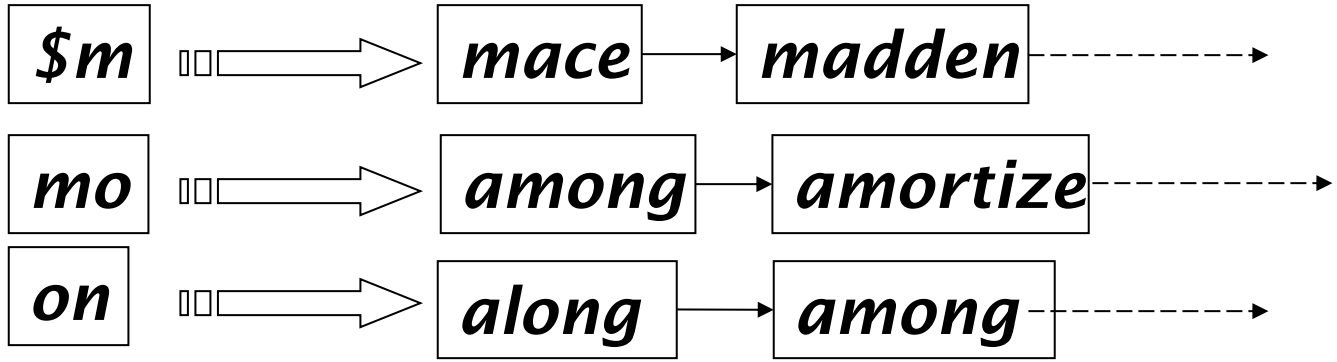
- Đối với truy vấn **h*a*o**, tìm kiếm **h*o** bằng cách tra cứu chuỗi **o$h***, sau đó thực hiện bước lọc lại: loại bỏ từ ***hello*** và giữ lại từ ***halo***.

Bigram (k-gram) Index

- Liệt kê **tất cả các k-gram** (chuỗi gồm **k ký tự liên tiếp**) xuất hiện trong mỗi thuật ngữ.
- Ví dụ từ câu "***April is the cruelest month***" ta thu được các **2-gram (bigram)**:
  - a, ap, pr, ri, il, l,
  - i, is, s,
  - t, th, he, e,
  - $c, cr, ru, ue, el, le, es, st, t$,
  - m, mo, on, nt, h
  - $ là ký tự đặc biệt đánh dấu ranh giới từ
- Ta xây dựng **một inverted index thứ hai**: bigram → các thuật ngữ trong từ điển chứa bigram đó, từ đó ta có thể **tìm các từ trong từ điển dựa trên các k-gram của chúng**.

Ví dụ Bigram Index

- **Chỉ mục k-gram** tìm các thuật ngữ dựa trên truy vấn gồm các **k-gram** (ở đây $k = 2$).

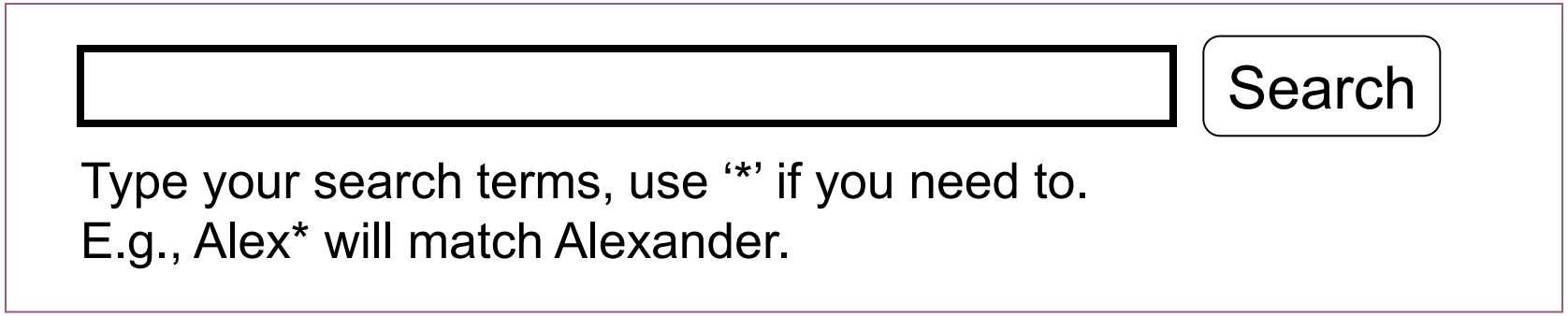


Xử lý wild-cards bằng k-gram

- Ví dụ truy vấn ***mon****
  - Có thể chuyển thành ***$m*** *AND* ***mo*** *AND* ***on***
- Lấy các **thuật ngữ khớp với phiên bản AND của truy vấn wildcards**.
- Tuy nhiên, cách này cũng có thể **liệt kê cả từ moon**.
- Vì vậy cần **lọc sau** các thuật ngữ này theo truy vấn ban đầu.
- Các thuật ngữ còn lại sau khi lọc sẽ được **tra cứu trong chỉ mục đảo**
- Phương pháp này **nhanh và tiết kiệm không gian (so với permuterm index)**.

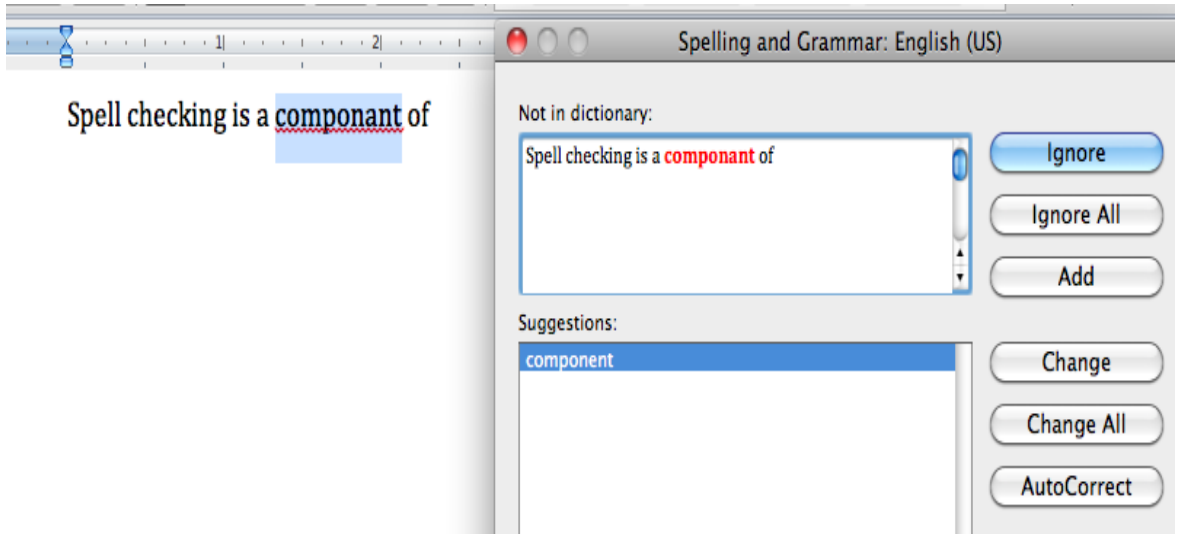
Xử lý truy vấn wild-card

- Như đã đề cập trước đó, chúng ta phải **thực thi một truy vấn Boolean cho mỗi thuật ngữ đã được liệt kê và lọc**.
- **Wild-cards** có thể dẫn đến **việc thực thi truy vấn tốn kém** (ví dụ: các phép **OR với số lượng rất lớn**).
  - pyth* AND prog*
- Nếu bạn **khuyến khích sự "lười biếng"** (cho phép dùng wild-card), **người dùng sẽ tận dụng điều đó!**



Các ứng dụng của sửa lỗi chính tả

Word processing



Phones

Web search

Showing results for natural ***language*** processing

Search instead for natural langage processing

Tỷ lệ lỗi chính tả

- Tùy theo ứng dụng, tỷ lệ lỗi có thể khoảng: ~1–20%
  - 26%: truy vấn Web — Wang et al., 2003
  - 13%: gõ lại văn bản, không dùng backspace — Whitelaw et al., English & German
  - 7%: các từ được sửa khi gõ lại trên thiết bị cỡ điện thoại
  - 2%: các từ không được sửa trên thiết bị organizer — Soukoreff & MacKenzie, 2003
  - 1–2%: gõ lại văn bản — Kane & Wobbrock, 2007; Gruden et al., 1983

Các nhiệm vụ trong sửa lỗi chính tả

- Phát hiện lỗi chính tả
- Sửa lỗi chính tả
  - Tự động sửa (Autocorrect)
    - hte → the
  - Đề xuất sửa lỗi
  - Danh sách gợi ý

Các loại lỗi chính tả

- Lỗi không phải từ hợp lệ (Non-word)
  - graffe → giraffe
- Lỗi từ hợp lệ (Real-word)
  - Lỗi đánh máy
    - three → there
  - Lỗi nhận thức / đồng âm
    - piece → peace
    - too → two
    - your → you're
- Việc **sửa lỗi** không phải từ hợp lệ trước đây thường **không phụ thuộc vào ngữ cảnh**.
- Trong khi đó, **sửa lỗi từ hợp lệ** hầu như **phải phụ thuộc vào ngữ cảnh**.

Lỗi chính tả dạng non-word

- Phát hiện lỗi non-word
  - Bất kỳ **từ nào không có trong từ điển** đều được coi là **lỗi**.
  - **Từ điển càng lớn càng tốt** … nhưng chỉ đến **một mức nào đó**.
  - (Trên Web có rất nhiều **từ viết sai**, vì vậy **Web không hẳn là từ điển tốt…**)
- Sửa lỗi non-word
  - Sinh các ứng viên
    - Các **từ hợp lệ** có **độ tương tự cao với từ sai**.
  - Chọn từ tốt nhất
    - Khoảng cách chỉnh sửa có trọng số ngắn nhất
    - Xác suất cao nhất theo mô hình noisy channel

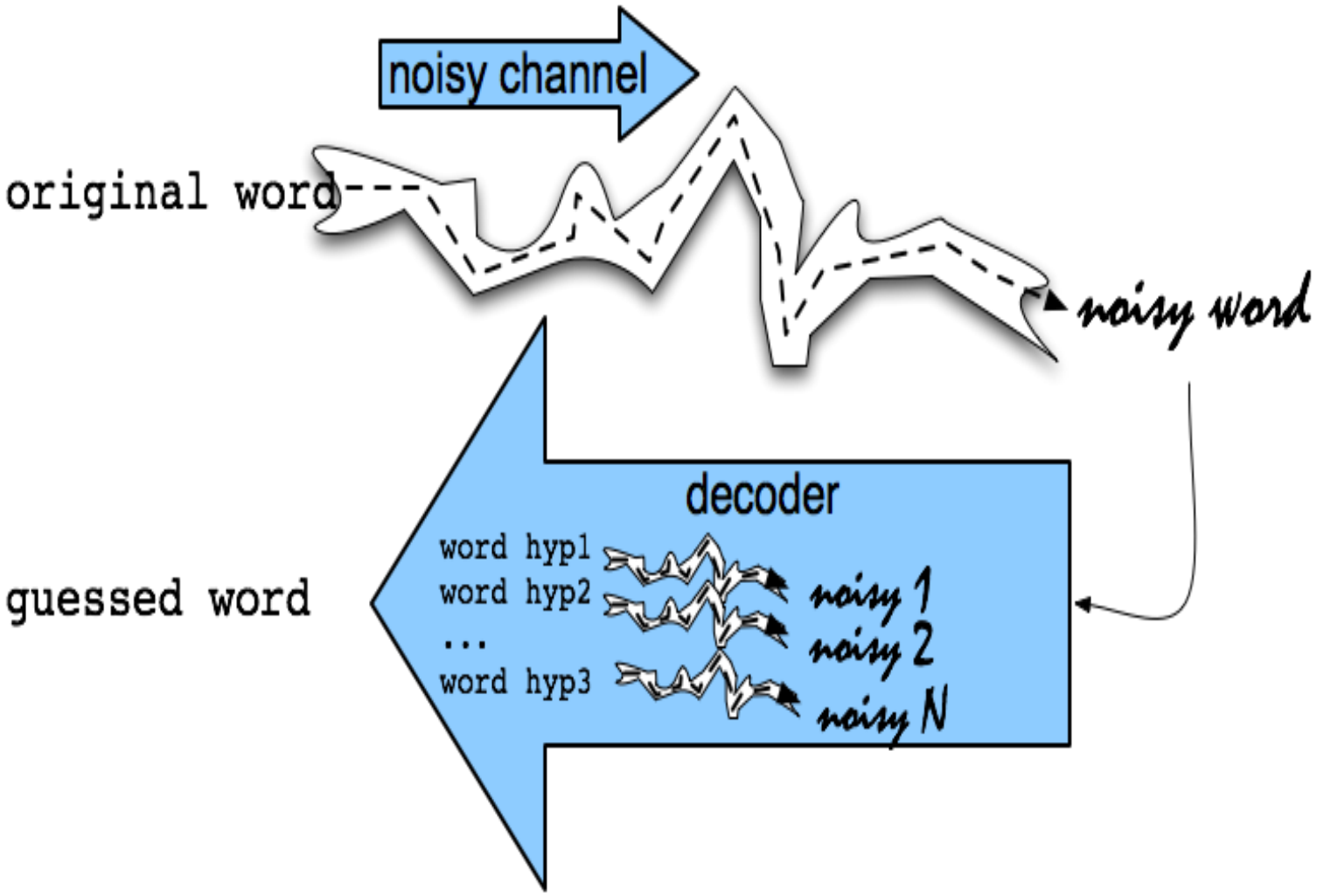
Lỗi chính tả dạng real-word và non-word

- Với mỗi từ **w**, tạo **tập các ứng viên**
  - Tìm các từ có **phát âm tương tự**.
  - Tìm các từ có **cách viết tương tự**.
  - **Bao gồm chính từ w** trong tập ứng viên.
- Chọn ứng viên tốt nhất.
  - Sử dụng **mô hình Noisy Channel** để mô hình hóa lỗi chính tả.
  - Phương pháp này **nhạy cảm với ngữ cảnh (context-sensitive)** nên cần xem xét liệu **các từ xung quanh có hợp lý hay không**.
  - *Flying <u>form</u> Heathrow to LAX → Flying <u>from</u> Heathrow to LAX*

Thuật ngữ

- Trước đó chúng ta đã nói về **character bigrams** và **k-grams**:
  - st, pr, an …
- Chúng ta cũng có thể có **word bigrams** và **n-grams**:
  - palo alto
  - flying from
  - road repairs

Trực giác về mô hình Noisy Channel



Mô hình Noisy Channel = Quy tắc Bayes

- Ta **quan sát một từ bị viết sai** x
- Mục tiêu là **tìm từ đúng** $\hat{w}$

$$\hat{w} = \underset{w \square V}{\operatorname{argmax}} P(w \mid x)$$

$$= \underset{w \square V}{\operatorname{argmax}} \frac{P(x \mid w)P(w)}{P(x)}$$

Bayes

$$= \underset{w \square V}{\operatorname{argmax}} P(x \mid w)P(w)$$

Prior

Noisy channel model

Lịch sử

- Mô hình Noisy Channel cho sửa lỗi chính tả được đề xuất khoảng năm 1990
- **IBM**
  - Mays, Eric, Fred J. Damerau and Robert L. Mercer. 1991. Context based spelling correction. *Information Processing and Management*, 23(5), 517–522
- **AT&T Bell Labs**
  - Kernighan, Mark D., Kenneth W. Church, and William A. Gale. 1990. A spelling correction program based on a noisy channel model. Proceedings of COLING 1990, 205-210

Ví dụ lỗi chính tả dạng non-word

acress

Sinh ứng viên

- Các từ có cách viết tương tự
  - → **khoảng cách chỉnh sửa nhỏ** so với từ sai
- Các từ có phát âm tương tự
  - → **khoảng cách phát âm nhỏ** so với từ sai

Kiểm tra ứng viên

- Dùng khoảng cách chỉnh sửa Damerau–Levenshtein
- **Khoảng cách chỉnh sửa tối thiểu** giữa hai chuỗi, trong đó các phép chỉnh sửa bao gồm các thao tác:
  - **Insertion** – thêm một ký tự
  - **Deletion** – xóa một ký tự
  - **Substitution** – thay thế một ký tự
  - **Transposition** – hoán đổi vị trí hai ký tự liền kề

Các từ có edit distance = 1 so với acress

Error	Candidate Correction	Correct Letter	Error Letter	Type
acress	actress	t	–	deletion
acress	cress	–	a	insertion
acress	caress	ca	ac	transposition
acress	access	c	r	substitution
acress	across	o	e	substitution
acress	acres	–	s	insertion

Sinh ứng viên

- **80%** lỗi nằm trong **edit distance = 1**
- **Gần như tất cả** lỗi nằm trong **edit distance = 2**
- Cũng cho phép **chèn thêm dấu cách hoặc dấu gạch nối**
  - `thisidea` → `this idea`
  - `inlaw` → `in-law`
- Cũng có thể cho phép **gộp từ**
  - `data base` → `database`
  - Với các văn bản ngắn như **truy vấn,** có thể xem **toàn bộ chuỗi như một mục** để sinh các phép chỉnh sửa.

Làm thế nào để sinh các ứng viên?

1. **Duyệt qua từ điển**, kiểm tra **edit distance** với từng từ.
2. **Sinh tất cả các từ có edit distance ≤ k** (ví dụ: $k = 1$ hoặc $k = 2$), sau đó **lấy giao với từ điển**.
3. **Sử dụng chỉ mục k-gram theo ký tự (character k-gram index)** để tìm các từ trong từ điển **chia sẻ nhiều k-gram nhất với từ đang xét.**
4. **Tính toán nhanh** bằng **Levenshtein finite state transducer**.
5. **Có thể dùng bảng ánh xạ được tính trước** từ các từ → các khả năng sửa lỗi.

Một mô hình…

- Chúng ta **muốn tìm các sửa lỗi chính tả tốt nhất**.
- Thay vì tìm **phương án tốt nhất tuyệt đối**, ta:
  - **Tìm một tập con các phương án khá tốt** (ví dụ: các từ có **edit distance ≤ 2**).
  - Sau đó **chọn phương án tốt nhất trong tập này**.
- Những phương án này **có thể không phải là phương án tốt nhất thực sự**, nhưng vẫn đủ tốt.
- Đây là **một mô hình lặp lại trong IR**:
  - tìm **các tài liệu tốt nhất cho một truy vấn**
  - tìm **câu trả lời tốt nhất**
  - tìm **quảng cáo tốt nhất…**
  - **…**
- Quy trình chung:
  1. Tìm một **tập ứng viên tốt**
  2. Chọn **top-K** trong số đó và trả về **kết quả tốt nhất**

Quay lại quy tắc Bayes

- Giả sử đã có các ứng viên
- Chúng ta **quan sát một từ bị viết sai** x
- Nhiệm vụ là **tìm từ đúng** $\hat{w}$

$$\hat{w} = \underset{w \square V}{\operatorname{argmax}} P(w \mid x)$$

$$= \underset{w \square V}{\operatorname{argmax}} \frac{P(x \mid w)P(w)}{P(x)}$$

$$= \underset{w \square V}{\operatorname{argmax}} P(x \mid w)P(w)$$

P(w) là gì?

Mô hình ngôn ngữ

- Lấy một **tập lớn các từ** (ví dụ: toàn bộ **tập tài liệu** của hệ thống) gồm **T token**.
  - Gọi *C(w)* = số lần xuất hiện của từ *w*
  - Khi đó, **xác suất của từ** *w* được ước lượng từ **tần suất xuất hiện của nó trong tập dữ liệu**.

$$P(w) = \frac{C(w)}{T}$$

- Trong các ứng dụng khác, nguồn dữ liệu cũng có thể là **tập các truy vấn mà người dùng đã gõ**

Xác suất tiên nghiệm Unigram

- Số liệu thống kê từ **404,253,213 từ** trong **Corpus of Contemporary English (COCA)**.

word	Frequency of word	P(w)
actress	9,321	0.0000230573
cress	220	0.0000005442
caress	686	0.0000016969
access	37,038	0.0000916207
across	120,844	0.0002989314
acres	12,874	0.0000318463

Xác suất của mô hình kênh

- Còn gọi là **xác suất mô hình lỗi** hoặc **xác suất chỉnh sửa**
- Tham khảo: **Kernighan, Church, Gale – 1990**.
- Giả sử:
  - **Từ viết sai** $x = x_1, x_2, x_3, \dots, x_m$
  - **Từ đúng** $w = w_1, w_2, w_3, \dots, w_n$
- Xác suất $P(x \mid w)$ là **xác suất từ đúng** w **bị chỉnh sửa thành từ sai** x thông qua các thao tác:
  - **Insertion** – thêm một ký tự
  - **Deletion** – xóa một ký tự
  - **Substitution** – thay thế một ký tự
  - **Transposition** – hoán đổi vị trí hai ký tự liền kề

Tính xác suất lỗi: ma trận nhầm lẫn

- **del[x,y]**: số lần **chuỗi xy được gõ thành x**
- **ins[x,y]**: số lần **x được gõ thành xy**
- **sub[x,y]**: số lần **y được gõ thành x**
- **trans[x,y]**: số lần **xy được gõ thành yx**

- **Insertion** và **deletion** được **điều kiện hóa theo ký tự đứng trước**.

Ma trận nhầm lẫn cho phép thay thế ký tự

sub[X, Y] = Substitution of X (incorrect) for Y (correct)

X	Y (correct)																									
	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
a	0	0	7	1	342	0	0	2	118	0	1	0	0	3	76	0	0	1	35	9	9	0	1	0	5	0
b	0	0	9	9	2	2	3	1	0	0	0	5	11	5	0	10	0	0	2	1	0	0	8	0	0	0
c	6	5	0	16	0	9	5	0	0	0	1	0	7	9	1	10	2	5	39	40	1	3	7	1	1	0
d	1	10	13	0	12	0	5	5	0	0	2	3	7	3	0	1	0	43	30	22	0	0	4	0	2	0
e	388	0	3	11	0	2	2	0	89	0	0	3	0	5	93	0	0	14	12	6	15	0	1	0	18	0
f	0	15	0	3	1	0	5	2	0	0	0	3	4	1	0	0	0	6	4	12	0	0	2	0	0	0
g	4	1	11	11	9	2	0	0	0	1	1	3	0	0	2	1	3	5	13	21	0	0	1	0	3	0
h	1	8	0	3	0	0	0	0	0	0	2	0	12	14	2	3	0	3	1	11	0	0	2	0	0	0
i	103	0	0	0	146	0	1	0	0	0	0	6	0	0	49	0	0	0	2	1	47	0	2	1	15	0
j	0	1	1	9	0	0	1	0	0	0	0	2	1	0	0	0	0	0	5	0	0	0	0	0	0	0
k	1	2	8	4	1	1	2	5	0	0	0	0	5	0	2	0	0	0	6	0	0	0	4	0	0	3
l	2	10	1	4	0	4	5	6	13	0	1	0	0	14	2	5	0	11	10	2	0	0	0	0	0	0
m	1	3	7	8	0	2	0	6	0	0	4	4	0	180	0	6	0	0	9	15	13	3	2	2	3	0
n	2	7	6	5	3	0	1	19	1	0	4	35	78	0	0	7	0	28	5	7	0	0	1	2	0	2
o	91	1	1	3	116	0	0	0	25	0	2	0	0	0	0	14	0	2	4	14	39	0	0	0	18	0
p	0	11	1	2	0	6	5	0	2	9	0	2	7	6	15	0	0	1	3	6	0	4	1	0	0	0
q	0	0	1	0	0	0	27	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
r	0	14	0	30	12	2	2	8	2	0	5	8	4	20	1	14	0	0	12	22	4	0	0	1	0	0
s	11	8	27	33	35	4	0	1	0	1	0	27	0	6	1	7	0	14	0	15	0	0	5	3	20	1
t	3	4	9	42	7	5	19	5	0	1	0	14	9	5	5	6	0	11	37	0	0	2	19	0	7	6
u	20	0	0	0	44	0	0	0	64	0	0	0	0	2	43	0	0	4	0	0	0	0	2	0	8	0
v	0	0	7	0	0	3	0	0	0	0	0	1	0	0	1	0	0	0	8	3	0	0	0	0	0	0
w	2	2	1	0	1	0	0	2	0	0	1	0	0	0	0	7	0	6	3	3	1	0	0	0	0	0
x	0	0	0	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0	9	0	0	0	0	0	0	0
y	0	0	2	0	15	0	1	7	15	0	0	0	2	0	6	1	0	7	36	8	5	0	0	1	0	0
z	0	0	0	7	0	0	0	0	0	0	0	7	5	0	0	0	0	2	21	3	0	0	0	0	3	0

Các phím gần nhau trên bàn phím

Tạo ma trận nhầm lẫn

- Danh sách lỗi của Peter Norvig
- Danh sách số lần xuất hiện của các lỗi chỉnh sửa một ký tự do Peter Norvig tổng hợp
- Tất cả các liên kết đến **dữ liệu n-gram của Peter Norvig**:

http://norvig.com/ngrams/

Mô hình kênh

$$P(x|w) = \begin{cases} \dfrac{\text{del}[w_{i-1}, w_i]}{\text{count}[w_{i-1} w_i]}, & \text{if deletion} \\ \dfrac{\text{ins}[w_{i-1}, x_i]}{\text{count}[w_{i-1}]}, & \text{if insertion} \\ \dfrac{\text{sub}[x_i, w_i]}{\text{count}[w_i]}, & \text{if substitution} \\ \dfrac{\text{trans}[w_i, w_{i+1}]}{\text{count}[w_i w_{i+1}]}, & \text{if transposition} \end{cases}$$

Làm trơn xác suất: Add-1 smoothing

- Nếu chúng ta sử dụng ví dụ **confusion matrix**, các lỗi **chưa từng xuất hiện sẽ có xác suất bằng 0**.
- Khi đó **xác suất tổng thể sẽ bằng 0**, điều này **quá khắt khe**.
- Ví dụ:
  - Trong bảng của **Kernighan**, các lỗi **q → a** và **a → q** đều có giá trị **0**, mặc dù **hai phím này nằm gần nhau trên bàn phím**.
- Giải pháp đơn giản:
  - Cộng **1** vào tất cả các giá trị đếm (**Add-1 smoothing**), sau đó **chuẩn hóa lại xác suất**.
  - Nếu bảng chữ cái có | A | ký tự, ta **chuẩn hóa tương ứng**.

$$\text{If substitution, } P(x \mid w) = \frac{\text{sub}[x, w] + 1}{\text{count}[w] + A}$$

Mô hình kênh cho từ acress

Candidate Correction	Correct Letter	Error Letter	x|w	P(x|w)
actress	t	–	c|ct	0.000117
cress	–	a	a|#	0.00000144
caress	ca	ac	ac|ca	0.00000164
access	c	r	r|c	0.000000209
across	o	e	e|o	0.0000093
acres	–	s	es|e	0.0000321
acres	–	s	ss|s	0.0000342

Xác suất Noisy Channel cho acress

Candidate Correction	Correct Letter	Error Letter	x|w	P(x|w)	P(w)	10^9 * P(x|w)*P(w)
actress	t	–	c|ct	0.000117	0.0000231	2.7
cress	–	a	a|#	0.00000144	0.000000544	0.00078
caress	ca	ac	ac|ca	0.00000164	0.00000170	0.0028
access	c	r	r|c	0.000000209	0.0000916	0.019
across	o	e	e|o	0.0000093	0.000299	2.8
acres	–	s	es|e	0.0000321	0.0000318	1.0
acres	–	s	ss|s	0.0000342	0.0000318	1.0

Bài tập

- Cho từ bị gõ sai:
  - acress
- Các từ ứng viên:
  - actress
  - across
  - access
  - acres
  - caress
- **Yêu cầu điền vào bảng**

Error	Candidate	Edit type	Edit distance
acress	actress	?	?
acress	across	?	?
acress	access	?	?
acress	acres	?	?
acress	caress	?	?
acress	actresss	?	?
acress	acerss	?	?

1. Xác định **loại lỗi (edit type)** giữa *acress* và mỗi ứng viên:
   - insertion
   - deletion
   - substitution
   - transposition
2. Tính **edit distance**.

Đánh giá

- Một số **bộ dữ liệu kiểm thử lỗi chính tả**:
  - Danh sách các lỗi viết sai phổ biến của Wikipedia
  - Phiên bản danh sách trên đã được Aspell lọc
  - Birkbeck spelling error corpus
  - Danh sách lỗi của Peter Norvig (bao gồm Wikipedia và Birkbeck, dùng cho huấn luyện hoặc kiểm thử)

Lỗi chính tả real-word

- …leaving in about fifteen ***minuets*** to go to her house.
- The design ***an*** construction of the system…
- Can they ***lave*** him my messages?
- The study was conducted mainly ***be*** John Black.

- Khoảng **25–40% lỗi chính tả** là **real words** *(Kukich, 1992)*.

Sửa lỗi chính tả phụ thuộc ngữ cảnh

- Với **mỗi từ trong câu** (cụm từ, truy vấn, …):
  - Sinh tập ứng viên gồm:
    - chính từ đó
    - **tất cả các chỉnh sửa một ký tự** mà tạo thành **từ tiếng Anh hợp lệ**
    - các từ đồng âm
    - (tất cả các bước này có thể được **tính trước**!)
- **Chọn ứng viên tốt nhất** bằng:
  - Mô hình Noisy Channel

Mô hình Noisy Channel cho sửa lỗi chính tả real-word

- Cho một câu: $x_1, x_2, x_3, \ldots, x_n$
- Sinh **tập ứng viên** cho mỗi từ x_i:
  - Candidate(x_1) = {x_1, w_1 , w'_1 , w''_1 ,…}
  - Candidate(x_2) = {x_2, w_2 , w'_2 , w''_2 ,…}
  - Candidate(x_n) = {x_n, w_n , w'_n , w''_n ,…}
- Chọn **chuỗi từ** W sao cho tối đa P(W|$x_1, \ldots, x_n$)

$$\hat{w} = \underset{w \square V}{\operatorname{argmax}} P(w \mid x)$$

$$= \underset{w \square V}{\operatorname{argmax}} P(x \mid w) P(w)$$

Kết hợp các từ ngữ cảnh: sửa lỗi chính tả nhạy cảm với ngữ cảnh

- Để xác định **actress** hay **across** là phù hợp, cần xem xét **ngữ cảnh sử dụng**.
- Điều này có thể thực hiện bằng **mô hình ngôn ngữ tốt hơn.**
- Một **bigram language model** giả định rằng **xác suất của một từ phụ thuộc vào từ đứng ngay trước nó**.

$$P(w_1 \ldots w_n) = P(w_1)P(w_2|w_1)\ldots P(w_n|w_{n-1})$$

Kết hợp các từ trong ngữ cảnh

- Với **unigram counts**, $P(w)$ **luôn khác 0** nếu từ điển được xây dựng từ **tập tài liệu**.
- Nhưng điều này **không đúng** với $P(w_k|w_{k-1})$. Vì vậy chúng ta cần **smoothing.**
- Có thể sử dụng **Add-1 smoothing** cho phân phối xác suất có điều kiện này.
- Tuy nhiên, có một cách tốt hơn - **nội suy giữa unigram và bigram**.

$$P_{li}(w_k|w_{k-1}) = \lambda P_{uni}(w_k) + (1-\lambda)P_{bi}(w_k|w_{k-1})$$

Trong đó: $P_{bi}(w_k|w_{k-1}) = C(w_{k-1}, w_k) / C(w_{k-1})$

Một số điểm quan trọng cần lưu ý

- Lưu ý rằng chúng ta có **nhiều phân phối xác suất khác nhau cho các từ**.
  - → Cần **phân biệt rõ ràng giữa chúng**.
- Có thể cần **làm việc với log-probability**:
  - $\log P(w_1 \ldots w_n) = \log P(w_1) + \log P(w_2 | w_1) + \ldots + \log P(w_n | w_{n-1})$
  - Nếu không, cần **cẩn thận với hiện tượng tràn số dưới.**
- Truy vấn của chúng ta có thể là **các từ xuất hiện ở bất kỳ vị trí nào trong tài liệu**.
  - Chúng ta sẽ **bắt đầu ước lượng bigram của một chuỗi bằng unigram estimate**.
  - Thông thường, người ta có thể **điều kiện hóa theo ký hiệu bắt đầu chuỗi,** nhưng **không phù hợp trong trường hợp này**.
  - Vì lý do này, **tổng số đếm của unigram và bigram sẽ khác nhau** — điều này **không phải là vấn đề**.

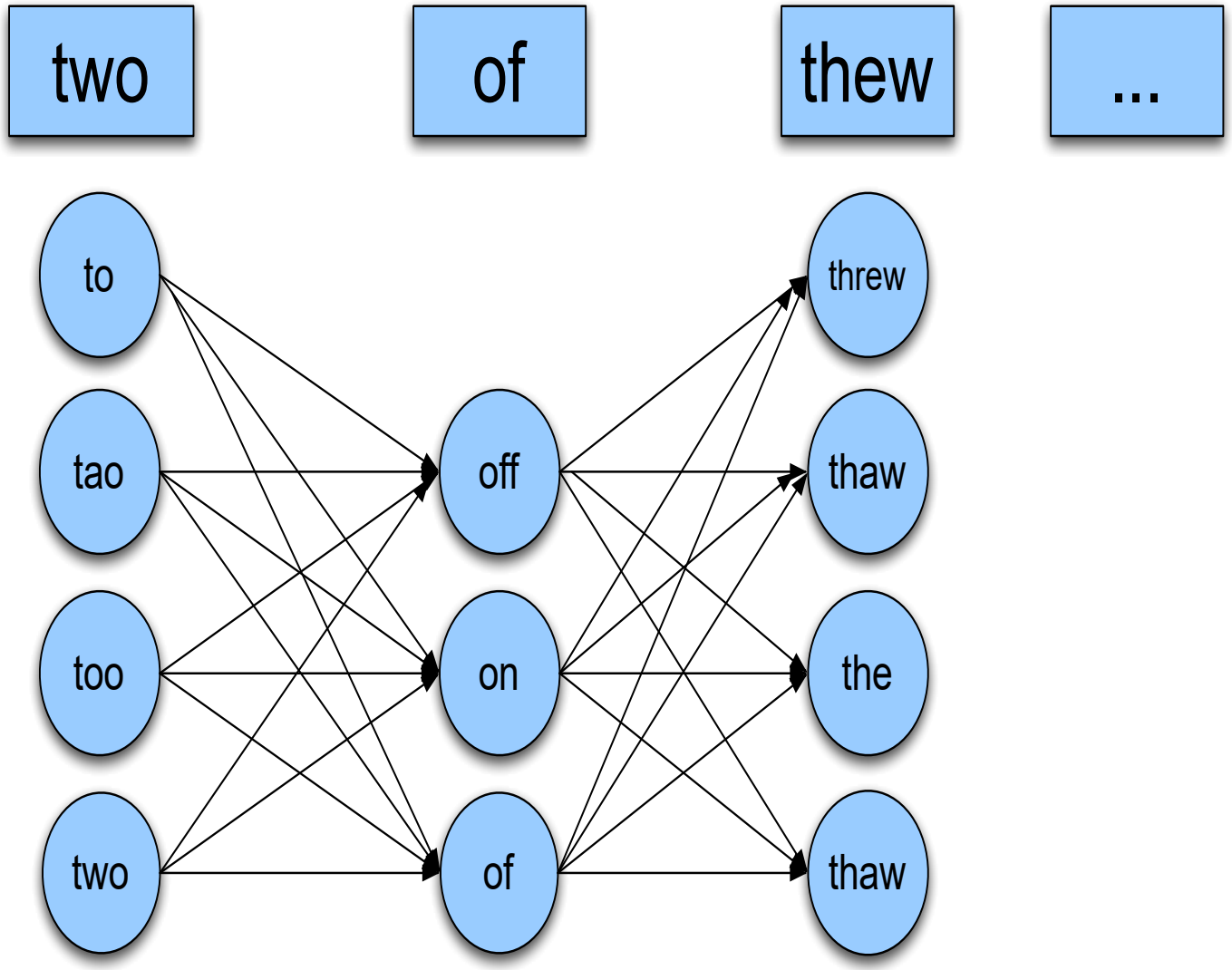
Sử dụng mô hình ngôn ngữ Bigram

- Ví dụ câu:
  - "a stellar and versatile **acress** whose combination of sass and glamour…"
- Các thống kê được lấy từ **Corpus of Contemporary American English**, với **add-1 smoothing**.
  - P(actress|versatile)=.000021
  - P(across|versatile) =.000021

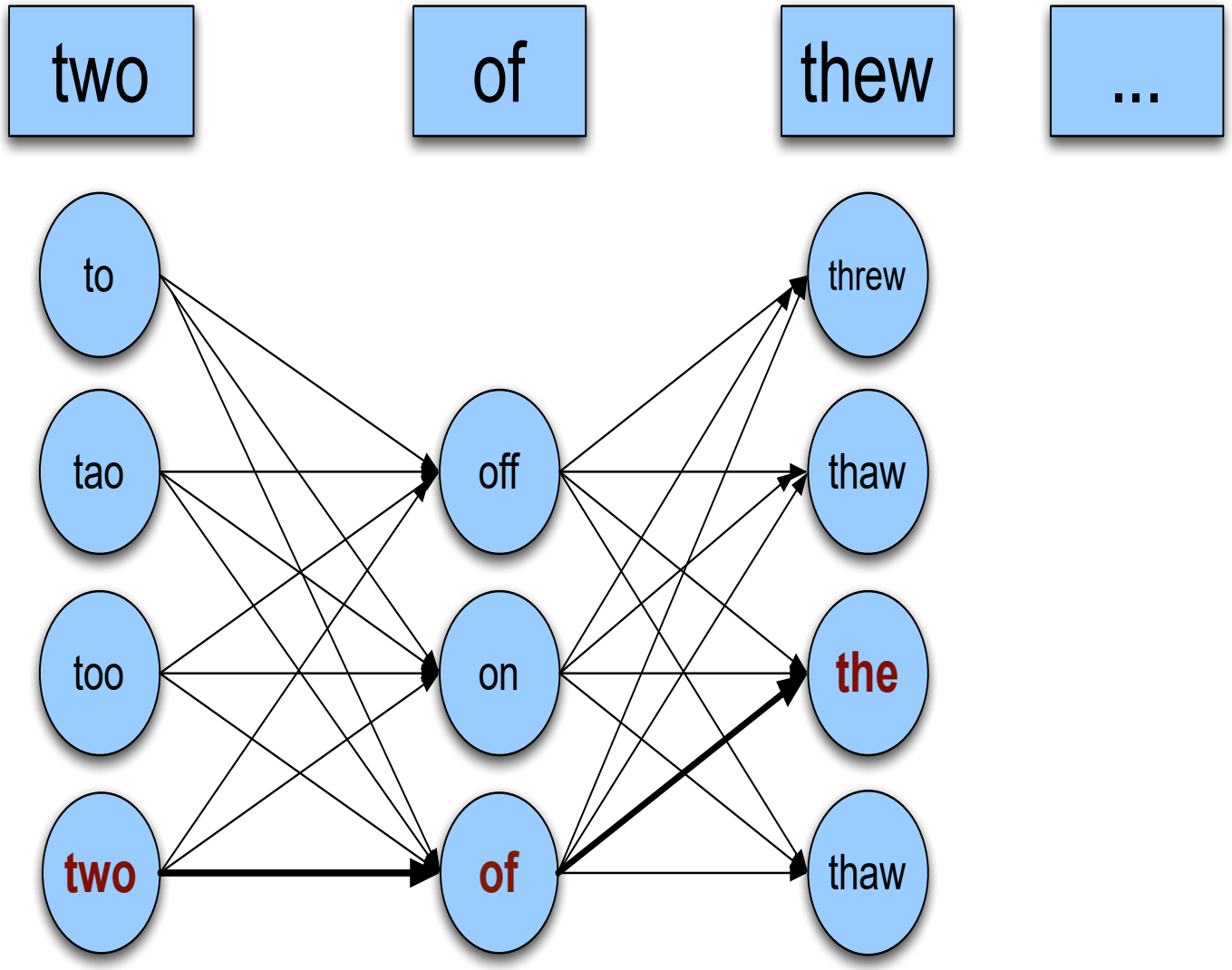
  P(whose|actress) = .0010
  P(whose|across) = .000006

  - P("versatile actress whose") = .000021*.0010 = 210 x10^{-10}
  - P("versatile across whose") = .000021*.000006 = 1 x10^{-10}

Mô hình Noisy Channel cho sửa lỗi chính tả real-word



Mô hình Noisy Channel cho sửa lỗi chính tả real-word



Đơn giản hóa: Mỗi câu chỉ có một lỗi

- Xét tất cả các **câu có thể có khi thay thế một từ trong câu**.
  - w_1, $\mathbf{w''_2}$,w_3,w_4 two **off** thew
  - w_1,w_2,$\mathbf{w'_3}$,w_4 two of **the**
  - $\mathbf{w'''_1}$,w_2,w_3,w_4 **too** of thew
  - …
- Chọn **chuỗi từ** W sao cho tối đa P(W)

Lấy các xác suất từ đâu

- **Language model**
  - Unigram
  - Bigram
  - v.v.
- **Channel model**
  - Giống như trong **sửa lỗi chính tả non-word.**
  - Ngoài ra cần **xác suất không có lỗi**: $P(w \mid w)$

Xác suất không có lỗi

- Xác suất của mô hình kênh khi **một từ được gõ đúng** là bao nhiêu?
- P(“the”|“the”)
  - Nếu có **một corpus lớn**, ta có thể **ước lượng tỷ lệ gõ đúng này**.
  - Tuy nhiên giá trị này **phụ thuộc mạnh vào ứng dụng**.
    - **0.90** → 1 lỗi trong 10 từ
    - **0.95** → 1 lỗi trong 20 từ
    - **0.99** → 1 lỗi trong 100 từ

Ví dụ “thew” của Peter Norvig

x	w	x|w	P(x|w)	P(w)	$10^9 \times P(x\|w)P(w)$
thew	the	ew|e	0.000007	0.02	144
thew	thew	–	0.95	0.00000009	90
thew	thaw	e|a	0.001	0.0000007	0.7
thew	threw	h|hr	0.000008	0.000004	0.03
thew	thwe	ew|we	0.000003	0.00000004	0.0001

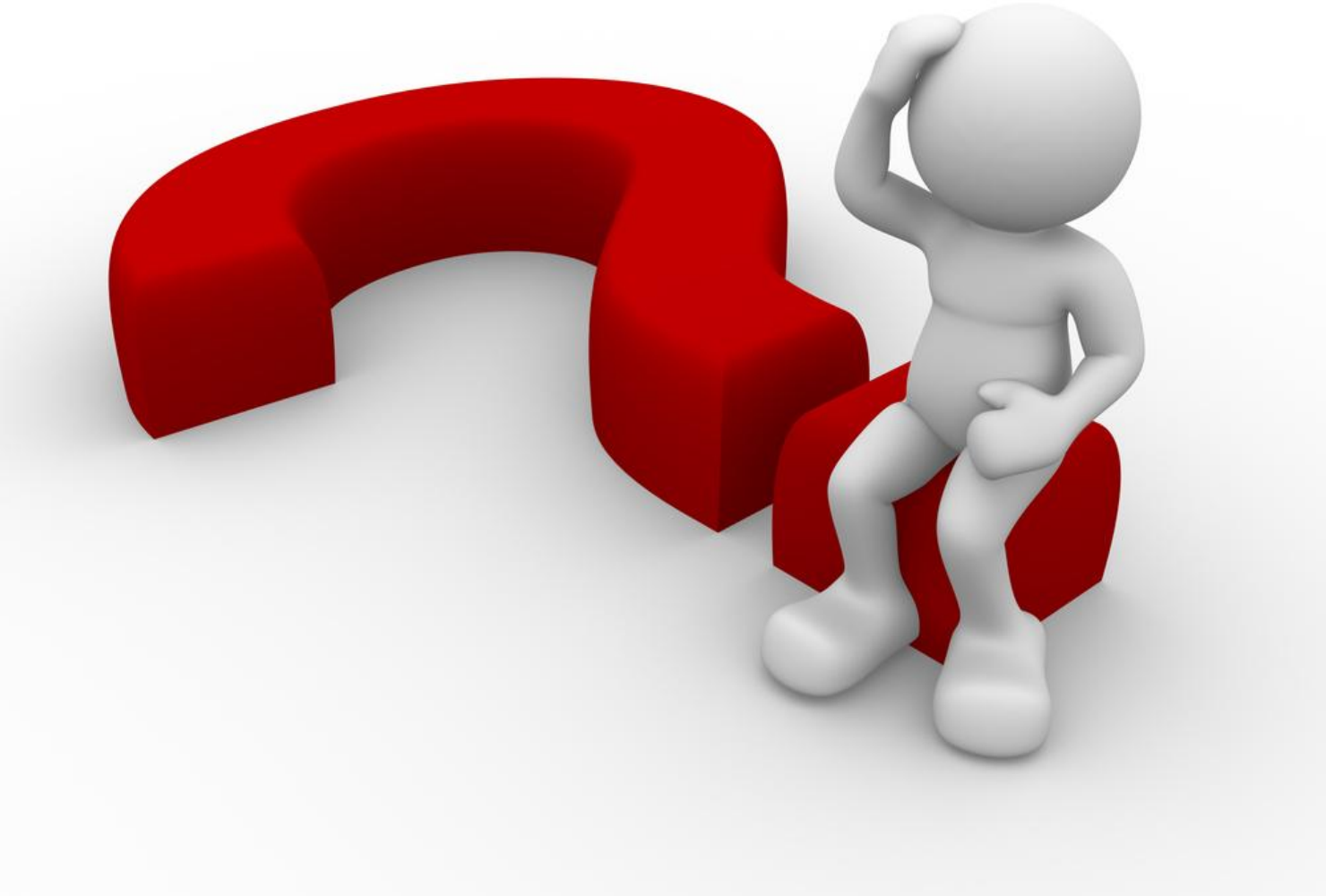
Mô hình Noisy Channel hiện đại

- Trong thực tế, **không chỉ đơn giản nhân trực tiếp xác suất xuất hiện từ trong ngôn ngữ và mô hình lỗi**.
- Do các **giả định độc lập**, các xác suất này **không hoàn toàn tương thích với nhau**.
- Vì vậy, thay vào đó người ta **gán trọng số cho chúng**.

$$\hat{w} = \underset{w \in V}{\operatorname{argmax}} P(x \mid w) P(w)^{\lambda}$$

- Giá trị λ được **học từ một tập phát triển**.

Cải tiến mô hình kênh

- Cho phép **các chỉnh sửa phức tạp hơn** (Brill and Moore 2000)
  - ent→ant
  - ph→f
  - le→al
- **Kết hợp phát âm** vào mô hình kênh (Toutanova and Moore 2002)
- **Kết hợp thông tin thiết bị** vào mô hình kênh
  - Ví dụ: các điện thoại Android khác nhau có thể có **mô hình lỗi khác nhau**
  - Tuy nhiên, **sửa lỗi chính tả có thể được thực hiện ở cấp độ hệ thống**.

Hỏi đáp